CSA1711 — ARTIFICIAL INTELLIGENCE

Analytical Questions and Solved Answers

SHABARISH N

192424007

Question 1: Informed Search (A* Algorithm)

Question: Consider a graph with nodes S, A, B, C, G. The edge costs are S–A = 1, S–B = 4, A–C = 2, B–C = 1, C–G = 3, A–G = 6. The heuristic estimates to goal G are h(S)=7, h(A)=6, h(B)=2, h(C)=3, h(G)=0. Using the A* algorithm, find the optimal path from S to G and its cost.

Solution:

A* selects nodes based on f(n) = g(n) + h(n), where g(n) is the actual cost from the start and h(n) is the heuristic estimate to the goal.

Path so far	g(n)	h(n)	f(n) = g+h
S	0	7	7
S–A	1	6	7
S–B	4	2	6
S–A–C	3	3	6
S–B–C	5	3	8
S–A–C–G	6	0	6
S–A–G	7	0	7

Expanding nodes in increasing order of f(n): S → A (f=7) and B (f=6) are generated; B has lower f but expanding B–C gives f=8, while S–A–C gives f=6, which is expanded next. From C, the path S–A–C–G gives f = 6 + 0 = 6, the lowest among all frontier nodes, so it is selected.

Optimal path: S → A → C → G

Optimal cost = 1 + 2 + 3 = 6

Question 2: Adversarial Search (Minimax with Alpha–Beta Pruning)

Question: A game tree has a MAX node at the root with three MIN children, each having three leaf values as follows: MIN1 leaves = {3, 5, 2}, MIN2 leaves = {9, 1, 7}, MIN3 leaves = {6, 8, 4}. Apply the minimax algorithm to find the value backed up to the root, and show which branches alpha–beta pruning would eliminate if leaves are evaluated left to right.

Solution:

Step 1 — Minimax values at MIN nodes (each MIN node takes the minimum of its children):

- MIN1 = min(3, 5, 2) = 2
- MIN2 = min(9, 1, 7) = 1
- MIN3 = min(6, 8, 4) = 4

Step 2 — MAX node takes the maximum of the MIN values: MAX = max(2, 1, 4) = 4.

Step 3 — Alpha–beta pruning (left to right):

- At MIN1: values 3, 5, 2 evaluated; MIN1 = 2. Alpha at root becomes 2 (best so far for MAX).
- At MIN2: first leaf 9 → current min 9; second leaf 1 → min drops to 1, which is ≤ alpha (2), so MIN2's remaining leaf (7) is pruned, since MAX will never choose a branch worth ≤ 1 over the guaranteed 2.
- At MIN3: leaves 6, 8, 4 evaluated normally (no pruning trigger since running min stays > alpha until the last leaf); MIN3 = 4, updating alpha to 4.

Final backed-up value at root = 4, with one leaf (the third value under MIN2) pruned.

Question 3: Knowledge Representation (Resolution in Propositional Logic)

Question: Given the knowledge base: (1) P → Q, (2) Q → R, (3) P. Using resolution refutation, prove that R is entailed by the knowledge base.

Solution:

Step 1 — Convert all statements to clausal (CNF) form:

- P → Q ≡ ¬P ∨ Q
- Q → R ≡ ¬Q ∨ R
- P (already a unit clause)
- Negate the goal: ¬R (added for refutation)

Step 2 — Apply resolution:

- Resolve P with (¬P ∨ Q) → gives Q
- Resolve Q with (¬Q ∨ R) → gives R
- Resolve R with ¬R → gives the empty clause (∅)

Since resolution derives the empty clause, the negated goal ¬R is contradictory with the knowledge base, so R is proved to be entailed.

Question 4: Reasoning Under Uncertainty (Bayesian Inference)

Question: A disease D affects 1% of a population. A test for D is 90% accurate for people who have the disease (true positive rate) and 95% accurate for people who do not have the disease (true negative rate). If a randomly chosen person tests positive, what is the probability that they actually have the disease? Use Bayes' theorem.

Solution:

Given: P(D) = 0.01, P(¬D) = 0.99, P(Positive | D) = 0.90, P(Positive | ¬D) = 0.05 (false positive rate = 1 − 0.95).

Bayes' theorem: P(D | Positive) = [P(Positive | D) × P(D)] / P(Positive)

Compute P(Positive) using the law of total probability:

P(Positive) = P(Positive|D)×P(D) + P(Positive|¬D)×P(¬D) = (0.90 × 0.01) + (0.05 × 0.99) = 0.009 + 0.0495 = 0.0585

P(D | Positive) = 0.009 / 0.0585 ≈ 0.1538

Result: There is only about a 15.4% chance the person actually has the disease, despite the positive test — illustrating the base-rate fallacy caused by the disease being rare.

Question 5: Local Search (Hill Climbing and Local Optima)

Question: A hill-climbing agent is searching a state space represented by the function f(x) = -(x-4)^2 + 20 - 5·sin(x), starting at x = 0 with step size 1, always moving to the neighbor (x-1 or x+1) with the higher f-value. Explain, with a worked comparison of neighbor values near x = 0 to x = 3, why the agent may fail to reach the global maximum, and name the specific problem encountered.

Solution:

Evaluate f(x) at integer points near the start:

x	-(x-4)^2	-5·sin(x)	f(x) approx
0	-16	0	4.00
1	-9	-4.21	6.79
2	-4	-4.55	11.45
3	-1	-0.71	18.29
4	0	3.78	23.78

From x=0, f increases steadily up to x=4 (f≈23.78), so simple hill climbing correctly moves 0→1→2→3→4. However, because f(x) includes the oscillating term -5·sin(x), for other starting points the function has local bumps and dips (ridges and small local peaks) where a neighboring step in either direction produces a lower f-value even though a better maximum exists further away.

If the agent starts at such a local peak, it stops because both neighbors score lower — this is the classic Local Maximum problem in hill climbing. Since basic hill climbing only accepts moves that strictly improve the current state and has no mechanism to look beyond immediate neighbors, it gets trapped and never reaches the true global maximum. Remedies include random-restart hill climbing, simulated annealing, or allowing sideways moves.

Question 6: Constraint Satisfaction Problem (Map Coloring)

Question: Four regions — W, X, Y, Z — must each be colored Red, Green, or Blue such that no two adjacent regions share a color. Adjacencies: W–X, W–Y, X–Y, X–Z, Y–Z. Using backtracking search with the constraint that W = Red is assigned first, find a valid color assignment for all regions.

Solution:

Adjacency constraints mean W, X, Y form a triangle (all mutually adjacent), and Z is adjacent to X and Y.

Step 1: Assign W = Red (given).

Step 2: X is adjacent to W, so X ≠ Red. Try X = Green.

Step 3: Y is adjacent to both W and X, so Y ≠ Red and Y ≠ Green. Y must be Blue.

Step 4: Z is adjacent to X (Green) and Y (Blue), so Z ≠ Green and Z ≠ Blue. Z must be Red.

Check all constraints: W(Red)-X(Green) ok, W(Red)-Y(Blue) ok, X(Green)-Y(Blue) ok, X(Green)-Z(Red) ok, Y(Blue)-Z(Red) ok — no violations.

Valid assignment: W = Red, X = Green, Y = Blue, Z = Red.

Question 7: Machine Learning (Entropy and Information Gain for a Decision Tree)

Question: A dataset of 10 examples for the target attribute 'PlayGame' has 6 'Yes' and 4 'No' instances. Splitting on attribute 'Weather' gives: Weather=Sunny (5 examples: 4 Yes, 1 No), Weather=Rainy (5 examples: 2 Yes, 3 No). Calculate the information gain of splitting on 'Weather' using entropy (log base 2).

Solution:

Step 1 — Entropy of the full dataset: Entropy(S) = -(6/10)log2(6/10) - (4/10)log2(4/10)

= -(0.6)(-0.737) - (0.4)(-1.322) = 0.442 + 0.529 = 0.971 bits

Step 2 — Entropy of Weather=Sunny (4 Yes, 1 No, total 5): -(4/5)log2(4/5) - (1/5)log2(1/5) = -(0.8)(-0.322) - (0.2)(-2.322) = 0.257 + 0.464 = 0.722 bits

Step 3 — Entropy of Weather=Rainy (2 Yes, 3 No, total 5): -(2/5)log2(2/5) - (3/5)log2(3/5) = -(0.4)(-1.322) - (0.6)(-0.737) = 0.529 + 0.442 = 0.971 bits

Step 4 — Weighted average entropy after split: (5/10)(0.722) + (5/10)(0.971) = 0.361 + 0.485 = 0.846 bits

Step 5 — Information Gain = Entropy(S) − Weighted Entropy = 0.971 − 0.846 = 0.125 bits

Information Gain of splitting on Weather ≈ 0.125 bits.

Question 8: Neural Networks (Forward Pass Calculation)

Question: A single-layer perceptron has two inputs x1 = 0.6 and x2 = 0.9, weights w1 = 0.4 and w2 = -0.3, and bias b = 0.1. Using the sigmoid activation function σ(z) = 1/(1+e^-z), calculate the output of the neuron.

Solution:

Step 1 — Compute the weighted sum (net input): z = (w1×x1) + (w2×x2) + b

z = (0.4 × 0.6) + (-0.3 × 0.9) + 0.1 = 0.24 − 0.27 + 0.1 = 0.07

Step 2 — Apply the sigmoid activation function: σ(0.07) = 1 / (1 + e^-0.07)

e^-0.07 ≈ 0.9324, so σ(0.07) = 1 / (1 + 0.9324) = 1 / 1.9324 ≈ 0.5175

Output of the neuron ≈ 0.5175 (about 51.75%), indicating the neuron is only marginally activated since the net input is close to zero.

Question 9: Intelligent Agents (Comparative Analysis of Agent Types)

Question: A self-driving delivery robot must navigate a warehouse to deliver packages while avoiding obstacles and adapting to new shelf layouts it has never seen before. Analyze which type of agent architecture — simple reflex, model-based reflex, goal-based, or utility-based — is the most suitable, and justify why the other three are insufficient.

Solution:

Analysis of each agent type against the requirements (novel layouts + obstacle avoidance + delivery goals):

- Simple Reflex Agent: Insufficient — it acts only on the current percept using condition-action rules, with no internal state. In a warehouse with unseen layouts, it cannot remember previously seen obstacles once out of view, leading to infinite loops or collisions.
- Model-Based Reflex Agent: An improvement — it maintains an internal state/model of the world, allowing it to track shelf positions and known obstacles even when not currently visible. However, it still only reacts to the current state; it has no explicit representation of 'delivering package to location B', so it cannot plan efficient multi-step routes toward a destination.
- Goal-Based Agent: Suitable — it maintains a model of the world and additionally has explicit goals (deliver package to a location), enabling it to search/plan a sequence of actions toward that goal, which is essential for navigating to a specific destination in a changing layout.
- Utility-Based Agent: Most suitable — it extends the goal-based agent with a utility function that ranks alternative paths not just by whether they achieve the goal, but by how well (e.g., shortest time, least

energy, safest route while avoiding obstacles). Since the robot must optimize between multiple valid paths under uncertainty, a utility-based architecture is best.

Conclusion: A Utility-Based Agent is the most suitable architecture, as it combines world-modeling (needed for unseen layouts), goal-directed planning (needed for delivery), and a utility measure to select the best among multiple obstacle-avoiding routes.

Question 10: Planning (STRIPS Representation)

Question: A robot arm must move Block A, which is currently On(A, Table) and Clear(A), onto Block B, which is Clear(B). The action MoveOnto(x, y) has Precondition: Clear(x) ∧ Clear(y), Effect (Add list): On(x,y); (Delete list): Clear(y), On(x,Table). Using STRIPS planning, write the initial state, goal state, and the resulting state after the action is applied.

Solution:

Initial State: { On(A, Table), Clear(A), Clear(B) }

Goal State: { On(A, B) }

Step 1 — Check preconditions for MoveOnto(A, B): Clear(A) ∧ Clear(B). Both hold in the initial state, so the action is applicable.

Step 2 — Apply the Add list: add On(A, B) to the state.

Step 3 — Apply the Delete list: remove Clear(B) and On(A, Table) from the state.

Resulting State: { On(A, B), Clear(A) }

Note: Clear(B) is correctly removed since B is now covered by A, and On(A, Table) is removed since A is no longer on the table. Clear(A) persists since nothing was placed on A. The goal On(A, B) is satisfied.

— End of Assignment —